CS584 – Assignment 3 (Discriminative Learning)

Due by: 3/16/2026

General Instructions

Objectives

By the end of this assignment, you will: (i) implement algorithms for discriminative learning (logistic regression, MLP, and deep networks); (ii) experiment with both tabular and image datasets; (iii) evaluate performance thoroughly using cross-validation and parameter studies; (iv) compare your implementations with standard baselines (e.g., scikit-learn, Keras).

- You must use two or more external datasets. At least one dataset should be images (e.g., digits, faces). Links to datasets are available on the course webpage (e.g., UCI Machine Learning Repository).
- It is essential to evaluate the performance of each algorithm and study the effect of varying parameters on performance.
- Use cross-validation to test performance unless otherwise specified.
- The grade will depend on both correctness of your implementations and thoroughness of evaluation.
- Your code must be modular, well documented, and written in Python.

Submission Instructions

- Upload your submission to Canvas.
- Submit a fully executed Python notebook with no gaps in execution. To receive full credit, please ensure: The notebook includes all cell outputs and contains no error messages. Each problem is clearly labeled in Markdown or comments (e.g., "Problem 2"). Re-run your notebook before submitting to ensure that cells are numbered sequentially starting at `[1]`.

Experiment Instructions

- Use at least two datasets: one tabular and one image dataset.
- Evaluate effects of varying hyperparameters (e.g., learning rate, hidden units, non-linear features).
- Compare results with `scikit-learn` and/or `Keras` baselines.
- Report standard performance metrics: confusion matrix, precision, recall, F1-score, accuracy.
- For parameter studies, use plots/tables with concise interpretations.

Programming Tasks

1. Logistic Regression

1. Implement logistic regression for two-class discrimination.
2. Extend your model by using non-linear combinations of inputs to increase classifier capacity.
3. Implement logistic regression for K-class discrimination.
4. Evaluate performance on multiple datasets.

2. Multilayer Perceptron (MLP)

1. Theory:
   - Consider a two-layer feedforward MLP with a single output unit where all elements (including the output) use sigmoid activation.
   - Training dataset: $x^{(i)}, y^{(i)}{}_{i=1}^{m}, \quad x^{(i)} \in \mathbb{R}^n, ; y^{(i)} \in 0, 1.$
   - Derive the backpropagation update equations for the weights of the output by minimizing the loss: $\frac{1}{2} \sum_{i=1}^{m} \left(y^{(i)} - \hat{y}^{(i)}\right)^2$.
   - Compare your derivation with the update obtained in class using maximum likelihood estimation.
2. Implementation:
   - Build a two-layer feedforward MLP for three-class classification.
   - Evaluate performance as a function of:
     - Number of hidden units
     - Learning rate parameter
     - Other hyperparameters (e.g., momentum)
   - Compare your implementation to the `scikit-learn` implementation.

3. Deep Learning with Keras

1. Use Keras to build and train networks on Google Colab (GPU required).
2. Load the IMDB sentiment classification dataset from Keras.
3. Split the dataset into training, validation, and test sets.
4. Vectorize the data using one-hot encoding.
5. Build a sequential classification network with several dense layers.
6. Compile and train the model.
7. Plot training and validation loss vs. epochs.
8. Plot training and validation accuracy vs. epochs.
9. Tune model hyperparameters (number of layers, hidden units, activation functions, loss functions) and retrain. Report evaluation on validation data.

4. Convolutional Neural Networks (CNNs)

1. Use Keras to build a convolutional neural network for image classification.
2. Choose an image dataset (e.g., MNIST digits, CIFAR-10).
3. Preprocess the dataset: normalize pixel values, split into training/validation/test sets, and apply categorical encoding for labels.
4. Design a CNN with the following elements: Convolutional layers, Pooling layers,Dense classification layers.

5. Compile and train the CNN.
6. Plot training and validation loss and accuracy as a function of epochs.
7. Perform a hyperparameter study: Vary number of convolutional filters. Try different kernel sizes and activation functions. Experiment with dropout for regularization
8. Evaluate final test performance and compare it to a baseline (e.g., simple fully connected network).

Reflection

At the end of your notebook, include a 1–2 paragraph reflection:

- Key insights gained from implementing discriminative learning methods
- Challenges and pitfalls encountered
- Lessons learned about practical training and evaluation of discriminative models

Starter Code

Below is some basic starter code for loading datasets, splitting into train/test, and evaluating models. Expand this for logistic regression, MLP, and deep networks.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, KFold
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score,
accuracy_score

# Example: load digits dataset (image data)
from sklearn.datasets import load_digits
digits = load_digits()
X, y = digits.data, digits.target

# Example: reduce to 2 classes for binary logistic regression
mask = (y == 0) | (y == 1)
X, y = X[mask], y[mask]

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Evaluation helper
def evaluate(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)
    prec = precision_score(y_true, y_pred, average='macro')
    rec = recall_score(y_true, y_pred, average='macro')
    f1 = f1_score(y_true, y_pred, average='macro')
    acc = accuracy_score(y_true, y_pred)
    return cm, prec, rec, f1, acc
```

```python
# Logistic Regression placeholder
def logistic_train(X, y, lr=0.01, epochs=1000):
    # TODO: implement training
    pass

def logistic_predict(X):
    # TODO: implement prediction
    pass

# MLP placeholder
def mlp_train(X, y, hidden_units=10, lr=0.01, momentum=0.9, epochs=1000):
    # TODO: implement MLP training with backpropagation
    pass

def mlp_predict(X):
    # TODO: implement prediction
    pass

# Keras example (IMDB)
from tensorflow import keras
from tensorflow.keras import layers

# Load dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.imdb.load_data(num_words=10000)

# Vectorize
x_train = keras.preprocessing.sequence.pad_sequences(x_train, maxlen=200)
x_test = keras.preprocessing.sequence.pad_sequences(x_test, maxlen=200)

# Build model
model = keras.Sequential([
    layers.Embedding(input_dim=10000, output_dim=32),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(x_train, y_train, epochs=5, batch_size=512, validation_split=0.2)
```